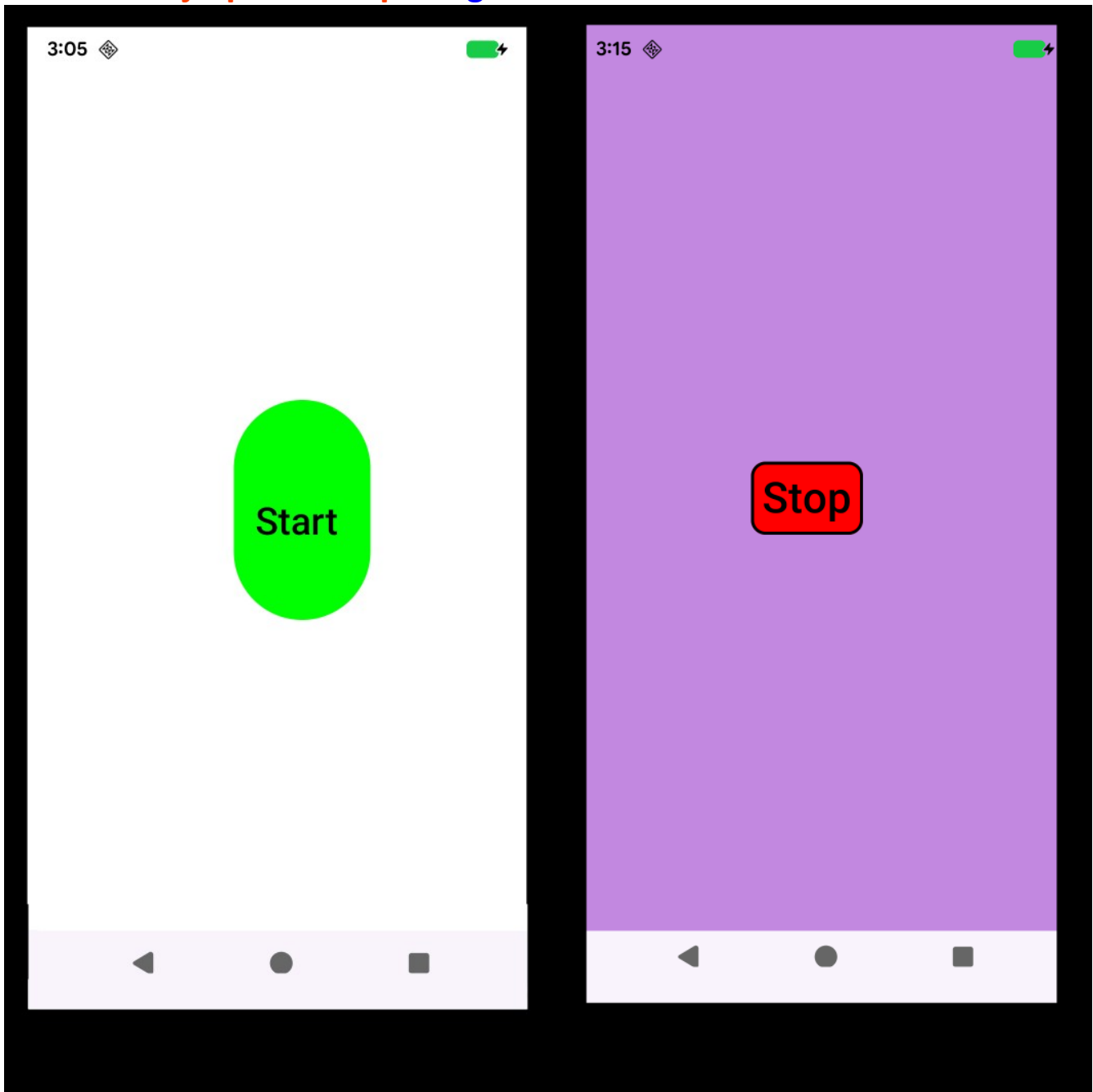


MVVM

This sample shows the proper way to create an **android smart phone app** that will not suffer from being too long on the ui thread, or having data scattered everywhere, let's follow the best **kotlin, mvvm** and **jetpack compose** guidelines.



If you press the **start button**, a background task, **not on ui thread**, changes the background color, every few seconds.

Table of Contents

MVVM.....	1
Quick Start.....	3
Clone Sample Project.....	4
Main Packages.....	7
ui core package.....	8
Main Files.....	10
The Basic Philosophy of the MVVM App.....	19
Program starts in onCreate.....	20
Pressing the start or stop button.....	21
ToggleRunning.....	22
Background Task Coroutine.....	23
Execute Background Task.....	24
Data Fetch Coroutine.....	25
One Time Data Fetch.....	26
Make Project your own.....	27
More Project “Code” details.....	32
Notes:.....	36

Quick Start

This guide explains how the android smart phone app,

MVVM, was designed, how it is expected to be used, by android smart phone app developers, such as yourself and how it works under the covers.

This app and guide, were not created to teach you kotlin, jetpack compose or mvvm. Those, and other necessary concepts, like how to install android and how to use github, are briefly covered, but there are many other guides to show and explain these things in greater detail.

This app **was designed** to help anyone use and understand how jetpack compose and mvvm apps are supposed to be designed or created.

It is expected that, after **renaming the sample, MVVM**, you will have the start of a well designed app that is also version controlled using [github](#).

Project MVVM has functionality and structure to help with common issues that all projects need, such as:

- communication between the user interface and the data being displayed, in such a way as to not do long running operations on the ui thread, which would lock up the app.
- Logging and other custom operations, via extension functions, [Extensions.kt](#)
- coroutine management
- [Prefs.kt](#) class to handle persisting data easily, when a database is not needed.
- Helpful, custom composeables [ui\composeables\core](#)
- repo repository to handle long running operations, such as fetching data from the web, other apps or just long running operations.

Clone Sample Project

It is preferable to use android studio to download project Mvvm from [github](#), but you may use [git](#) directly to download the project files and the apk from [url: https://github.com/pstorli/Mvvm.git](https://github.com/pstorli/Mvvm.git)

The apk file [Mvvm.apk](#), in the root of the project directory, has already been compiled for you, in case you have not yet installed Android Studio, and can be [side loaded](#) to your android phone.

The file [Guide.pdf](#), also in the project root dir, is [this file](#).

a) **Open Android Studio.**

If you don't have android studio, download and install it from:

<https://developer.android.com/studio>

This sample project was built using:

Otter 2 feature Drop

b) **File → Close Project**

Do above action for all open projects.

c) Press the **“Clone Repository”** button.

d) Enter [url](#) and [destination](#) directory.

Note: [Parent Directory](#) does not have to be: C:\github
and **Name** does not have to be [MySample](#),
use any name for your new project, just no spaces.

Enter the following:

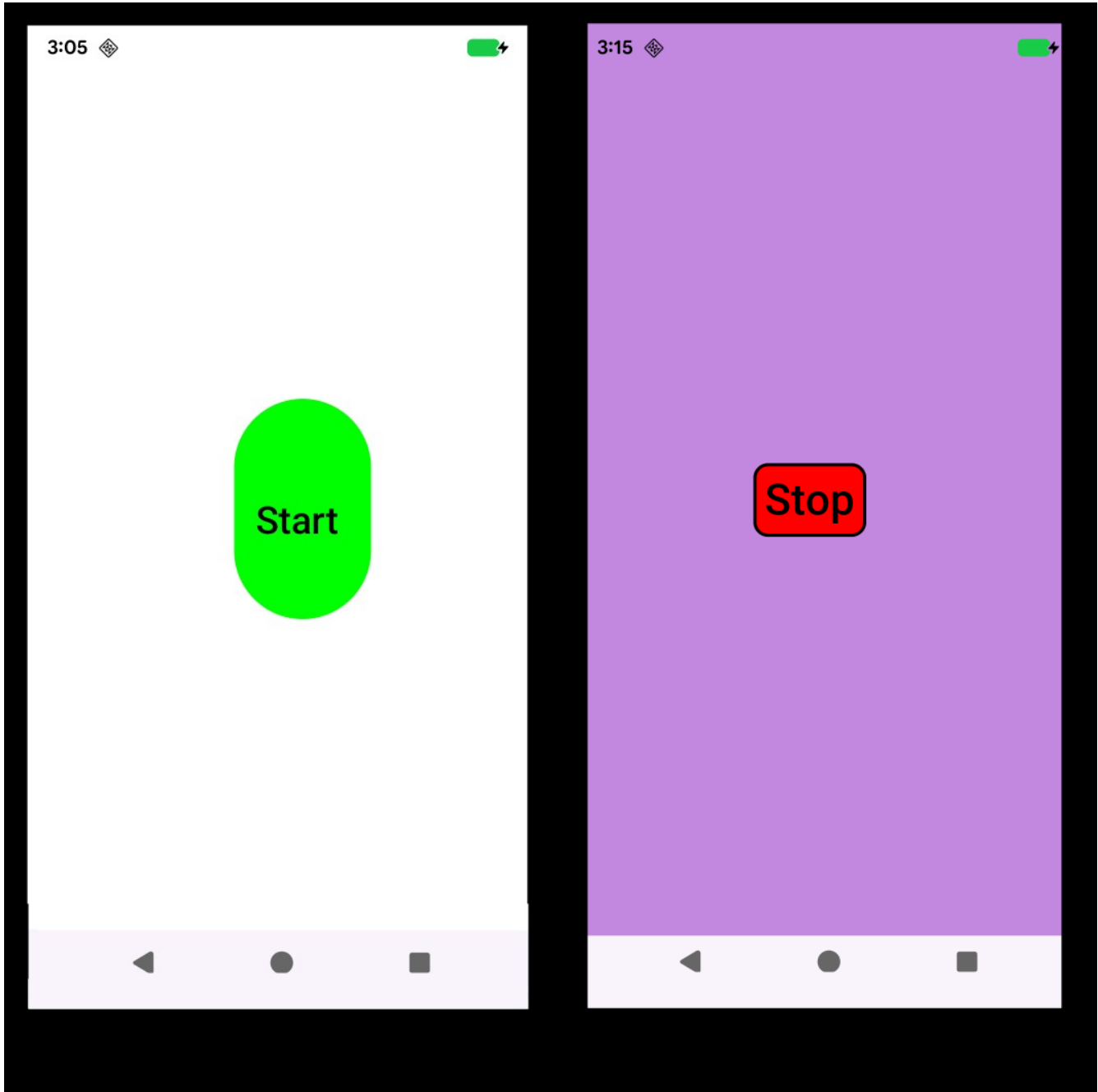
URL: <https://github.com/pstorli/MVVM.git>

Directory: C:\github\MySample

e) **Run → Run 'app'**

Connect phone with USB cable or use emulator.

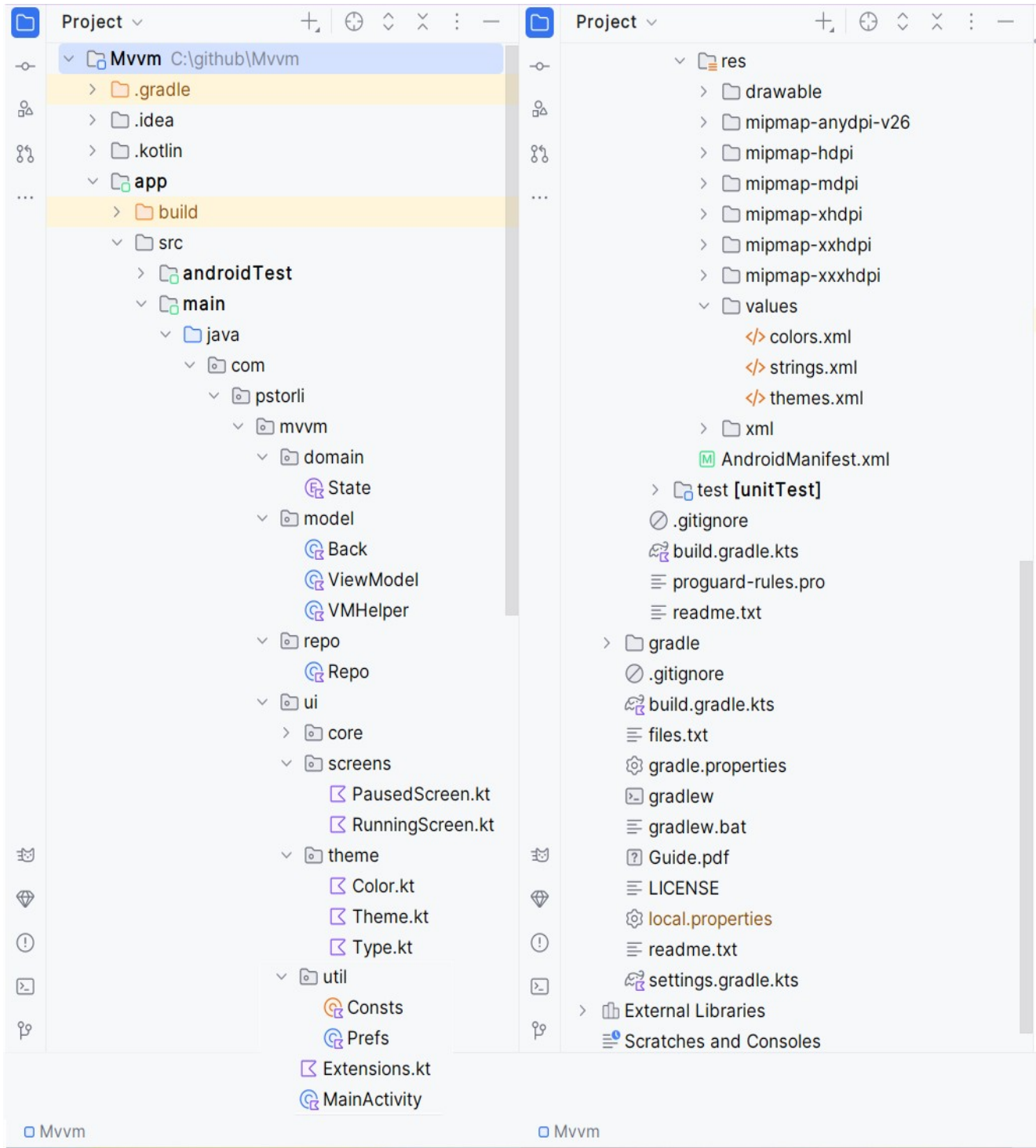
Either way, app should look like this:



The button should toggle between paused and running.

f) **File** → **Close Project**. The next section covers [project architecture](#), [program flow](#), [renaming project](#) and adding **it** to your own [github](#) account.

Android Studio Project Diagram

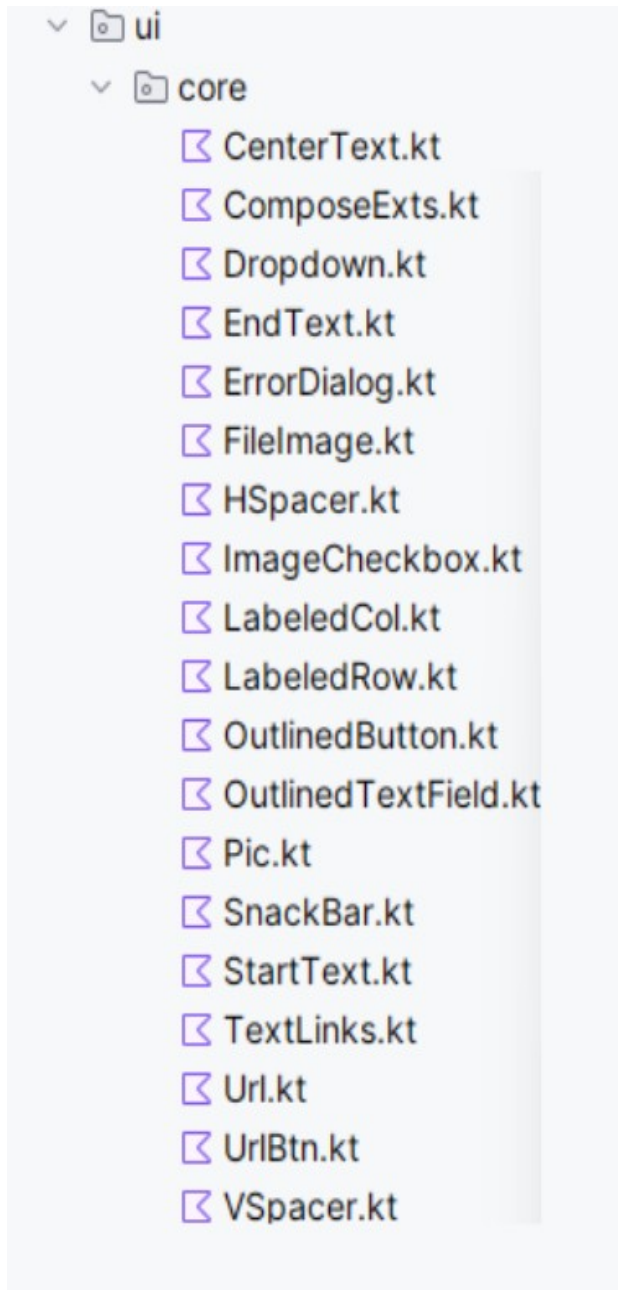


Main Packages

- **Domain package:** proj data. Currently we just have an enum class,
State.kt – *PAUSED* and *RUNNING*
- **model package:**
handles data src and ui / viewmodel communication.
Back.kt class handles viewmodel /repo communication, via coroutines, used to perform long running operations, that would otherwise **lock up the ui thread**.
ViewModel.kt class has **viewmodel** which is data source, holds **repo** and **prefs**
VMHelper.kt class holds viewmodel logic.
- **repo package**
Repo.kt class for long running functions, done **not on ui thread**!
- **res package** holds resources, such as images and text strings
drawable package Holds project image files, **png** and **jpg**
values package **Strings.xml** file holds user displayable text, one file per language

• ui core package

holds custom, generic, composable classes for project



Composables are easy to use and make. Look at these examples in this directory for project independent composableables.

Pay careful attention to how the composable is created, parameters passed, and how it indicates action, such as what a button press or click does.

The composable, **OutlinedButton.kt**, is used as the sample button in the composable **RunningScreen.kt**.

Copy a composable in core, play around with it, make new ones, they are re-usable gui, kind of like a reusable class in kotlin.

- **ui package**

core package Custom, Generic, composeables

screens package holds project's compose screens

PausedScreen.kt A composeable sample start screen to show when in **Paused** state.

RunningScreen.kt – A composeable to show when in **Running** state.

theme package

Theme.kt A composeable that handles **light** and **dark** color schemes.

- **util package holds utility classes**

Prefs.kt class used for persisting data, when a database is overkill.

Consts.kt class used for project wide constants.

Main Files

.gitignore

```
/**  
 * This file has Files to not save into version control  
 */
```

build.gradle.kts

// The namespace property defines the package name for your app's

// **internal generated code** like the R class

// (for resources) and BuildConfig class.

namespace = "com.pstorli.mvvm"

// **unique identifier** for your application on an Android device

applicationId = "com.pstorli.mvvm"

// **What min and max android version do you support?**

minSdk = 24

targetSdk = 36

// The version this app is on.

versionCode = 1

versionName = "1.0"

// This section is used to import external projects we need.

dependencies {}

Back.kt

```
/**  
 * Back for run in background, not on ui thread.  
 *  
 * This class is used to spin off co-routines which call  
 * long running functions in the Repo.kt class  
 * NOTE: The repo class is private, we don't want it being  
 * used by anyone but this class, which knows how to launch  
 * co-routines.  
 */
```

Color.kt

```
/**  
 * This class holds the colors used in the app.  
 */
```

colors.xml

```
/**  
 * Hex color codes.  
 */
```

Consts.kt

```
/**  
 * A place to put project constants.  
 */
```

Extensions.kt

```
/**  
 * A place to put class extensions.  
 */
```

A class extension allows you to add functionality to a class that it does not have. For example to add easy exception logging,

fun, then, the class, a period, then the function namespace

Like this:

```
fun Exception.logError ()  
{  
    Consts.logError (this.toString())  
}
```

```
// This is how we use it.  
catch (ex: Exception) {  
    ex.logError()  
}
```

MainActivity.kt

/**

This class is the applications main activity.

This is the only place we instantiate the **ViewModel.kt** class.

It is passed to composeables on their constructor.

The function, **onCreate**, in **MainActivity.kt**

override fun onCreate(savedInstanceState: Bundle?) {}

is called when program starts up and is on the ui thread, so no long running operations here, just call setContent to set the composeable to view,

In this case, **PausedScreen.kt**

*/

PausedScreen.kt

```
/**
```

This is an example of a typical composable.
Note that the viewModel is passed in on the constructor.

```
// This line both sets the background color, and listens for  
// any future changes in the background color in the viewModel
```

```
.background(viewModel.backgroundColor),
```

```
OutlinedButton (
```

```
    name      = viewModel.btnText,  
    backgroundColor = Color (viewModel.btnBackColor.value),  
    borderColor = Color.Red,
```

```
    // Toggle running state when clicked
```

```
    onClick = {  
        // Tell the view model to toggle the running state.  
        viewModel.vh.toggleRunning()  
    })
```

The **toggleRunning()** call, tells the viewModel that we want to toggle the running state. The ui should only handle two things, displaying the data and notifying the viewModel of ui actions.

We want to keep logic out of the ui, as much as possible.

```
*/
```

Prefs.kt

```
/**  
 * This class can be used to store and retrieve data that persists, even when  
 app and phone are turned off. The only way to “clean” or “clear” the  
 preference data is by un-installing app.  
 */
```

readme.txt

```
/** A place to store info about developer, project and version changes.  
0000 Changes for Version X:  
Changed ...  
*/
```

Repo.kt

```
/** This class handles talking to remote data sources to get data in the  
background, such as images from urls, websites, vpn, or even local database ,  
which is considered slow.
```

This class is currently designed to be called from class Back.kt
so that it is never called directly from the ui thread.

If request is needed, add methods to Back.kt to get to the Repo.kt method you
want to talk to.

The function `oneTimeDataFetch`, normally does not need to be here, it is not long running.

But it have it here as an example of how a longer running routine should be handled to fetch some data.

```
/**  
 * Fetch a color.  
 */  
fun oneTimeDataFetch(): Color
```

The function `executeBackgroundTask`, is called repeatedly, in the background, off the ui thread.

```
/**  
 * This function is called repeatedly, with delay, Consts.kt.  
 * the background task,  
 * which keeps running, periodically, between  
 * Consts.MIN_DELAY and Consts.MAX_DELAY until  
 * viewModel.state is PAUSED  
 */  
fun executeBackgroundTask() {
```

RunningScreen.kt

/**

This is an example of a typical composable.
Note that the viewModel is passed in on the constructor.

This screen is shown when app in running state.

strings.xml

/**

Where to put strings being displayed to the user.
Useful for internationalization.
Strings used for logging need not go here.

*/

Theme.kt

/**

* Handles switching between dark and light modes for colors.

*/

themes.xml

/**

* *Where to declare theme to use.*

*/

Type.kt

/**

** Typography, font family, font size, font weight, line height and spacing*

*/

ViewModel.kt

/**

Holds project data, in a way that survives context switches,
such as changing the phone's orientation

*/

VMHelper.kt

/**

* This class is used for logic that is short lived, as we are on the ui thread here. The reason is that I like to keep the **viewModel** as solely a data repository, and as little logic, functions, as possible.

*/

The Basic Philosophy of the MVVM App

MVVM and **Jetpack Compose** aim to make programming easier by providing way to have data available from a central location, **ViewModel.kt**, and to help with keeping long running operations off the UI thread, **Back.kt**

There are **three main events** that I would like to cover. What happens at **program startup**, what happens when the **start button** is pressed, and what happens when the **stop** button is pressed.

*Hint: Both call **toggleRunning**.*

Program starts in onCreate.

All android programs start in **MainActivity.kt**

```
override fun onCreate(savedInstanceState: Bundle?)
```

The main thing to do here is create the correct composable, inject it with the **viewModel** and set the content.

The **viewModel** holds the **data** for the entire app.

```
val viewModel: ViewModel by viewModels()
```

The content **should** be a **composable**.

We pass the **viewModel** to the composables constructor.

MvvmTheme.kt class handles colors the app uses in normal and dark mode.

```
setContent {  
    MvvmTheme {  
        Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
            // Show correct screen for game state.  
            when (viewModel.state) {  
                PAUSED ->  
                    PausedScreen (  
                        viewModel = viewModel,  
                        modifier = Modifier.padding(innerPadding)  
                    )  
                RUNNING ->  
                    RunningScreen (  
                        viewModel = viewModel,  
                        modifier = Modifier.padding(innerPadding)  
                    )  
            }  
        }  
    }  
}
```

Pressing the start or stop button.

The logic to handle the button press is in class **PausedScreen.kt** and **RunningScreen.kt**

```
Button (  
    // Toggle running state when clicked  
    onClick = {  
        viewModel.vh.toggleRunning()  
    },
```

When the user clicks the Start or Stop button, the routine **toggleRunning** is called in class **VMHelper.kt** to change the **viewModel.state**

We first update the state in class **ViewModel.kt**

Note: The **State.kt** class is enclosed by class **mutableStateOf**

```
var state by mutableStateOf<State>(State.PAUSED)
```

which, when modified, notifies the routine **onCreate**, in **MainActivity.kt**

```
when (viewModel.state)
```

With the following code in class **VMHelper.kt** :

```
// toggle running state  
if (RUNNING == viewModel.state) {  
    viewModel.state = PAUSED  
}  
else {  
    viewModel.state = RUNNING  
}
```

ToggleRunning

The next thing we do in the routine **toggleRunning** in class **VMHelper.kt** is call either **backgroundTaskCoroutine** or **dataFetchCoroutine** in class **Back.kt**

```
if (RUNNING == viewModel.state) {  
    back.backgroundTaskCoroutine()  
}  
else {  
    back.dataFetchCoroutine ()  
}
```

Background Task Coroutine

The routine **backgroundTaskCoroutine** in class **Back.kt** causes a background task, a co-routine, to start in view model scope:

```
bj = viewModel.viewModelScope.launch
```

The thread, co-routine, will keep running as long as the state is Running.

```
while (RUNNING == viewModel.state)
```

It repeatedly calls:

```
repo.executeBackgroundTask()
```

With a delay between calls.

```
// Wait 1 - 5 seconds
```

```
val time = (Consts.rndNum(MIN_DELAY, MAX_DELAY)).toLong()  
"Back.backgroundTaskCoroutine Delaying: $time".logVerbose()
```

```
delay (time)
```

Note: The **logVerbose** call only logs if **Consts.kt VERBOSE** is true.
Use **logInfo** to always log events.

Execute Background Task

The routine **executeBackgroundTask** in class **Repo.kt** can do anything you want. It is called from the routine **backgroundTaskCoroutine** in class **Back.kt**

```
fun executeBackgroundTask()
```

It currently creates a new color,

```
val color = randomColor ()
```

and then sets the background color to the new color.

```
viewModel.backgroundColor = color
```

Data Fetch Coroutine

The routine **dataFetchCoroutine** in class **Back.kt** causes a background task, a co-routine, to start in view model scope:

```
viewModel.viewModelScope.launch
```

It calls the routine **oneTimeDataFetch** in **Repo.kt** one time only.

```
// Create deferred var
val fetchColorDeferred = viewModel.viewModelScope.async
(Dispatchers.Main)
{
    repo.oneTimeDataFetch ()
}

// Wait for it. new button color
val color = fetchColorDeferred.await()

// Go back contains ui thread.
// This causes us to rejoin the ui thread,
withContext (Dispatchers.Main) {

    // Update the button color in the view model.
    viewModel.backgroundColor = color
}
```

One Time Data Fetch

The routine **oneTimeDataFetch** in class **Repo.kt** can do anything you want. It is called from the routine **dataFetchCoroutine** in class **Back.kt**

```
fun oneTimeDataFetch() : Color
```

It currently creates a new color,

```
val color = randomColor ()
```

and then returns the new color to the **dataFetchCoroutine** in class **Back.kt**

```
return color
```

Make Project **your own.**

Using this app, this sample project, as the foundation for any Android App that you want to create is kind of like buying a furnished house with plumbing and electricity versus what you get when you say “New Project” in Android studio, a cheaper house that has only walls and a roof, no plumbing, electricity or furniture!

a) Open a file explorer and go to the location where you downloaded the “MySample” application,

For me it was dir: C:\github\MySample

b) Delete the **.git** directory.

It is hidden, so you may need to show hidden files and dirs, in your file explorer.

c) Adjust the strings XML file.

1) Open the strings XML file, in a text editor

*For me it was dir: C:\github\MySample
\app\src\main\res\values\strings.xml*

2) Find the string resource typically named **app_name** and change its value to your new app name.

a) **Is:**

`<string name="app_name">MVVM</string>`

b) **change to:**

`<string name="app_name">MySample</string>`

d) Update the **build.gradle** file. You must update the **application ID** in your module's build.gradle file.

Locate build.gradle. My gradle file was at:

C:\github\MySample\app\build.gradle.kts

Modify applicationId. Find the android block and update the applicationId to your new, full package name.

Before:

```
android {  
    namespace = "com.pstorli.mvvm"  
    compileSdk {version = release(36)}  
  
    defaultConfig {  
        applicationId = "com.pstorli.mvvm"  
        minSdk = 24  
        targetSdk = 36  
        versionCode = 1  
        versionName = "1.0"
```

After:

```
android {  
    namespace = "com.mycompany.mysample"  
    compileSdk {version = release(36)}  
  
    defaultConfig {  
        applicationId = "com.mycompany.mysample"  
        minSdk = 24  
        targetSdk = 36  
        versionCode = 1  
        versionName = "1.0"
```

e) Check the **AndroidManifest.xml**

While the steps above usually handle everything, it's good practice to ensure the package attribute in your **AndroidManifest.xml** file is pointing to the new package name.

NOTE: If you are happy with '**Current**' being the **theme name**, skip this step.

1. Locate AndroidManifest.xml:

Open up file:

C:\github\MySample\app\src\main\AndroidManifest.xml

2. Verify the package: Make sure the package attribute at the top of the file reflects your new package name.

Change, in two places, from:

android:theme="@style/Theme.Current"

to:

android:theme="@style/Theme.MySample"

f) Adjust the idea workspace file for your project.

1) Open up file:

C:\github\MySample\.idea\workspace.xml

2) Replace

"MVVM" with **"MySample"**

3) Then, Replace

"pstorli" with **"mycompany"**

f) Adjust the, hidden, idea name file for your project.

Note: If file does not exist, skip this step,

but your file explorer may be hiding it from you.

1) Open up file:

C:\github\MySample\.idea\.**name**

2) Replace

“**MVVM**” with “**MySample**”

g) Rename directories. *Note:* **mycompany**, **mysample** and **MySample**, are just suggested, names should be **case sensitive** and **have no spaces or other special characters**, so use whatever names you like.

1) Rename project name directory:

from:

C:\github\MySample\app\src\main\java\
com\pstorli\mvvm

to:

C:\github\MySample\app\src\main\java\
com\pstorli\mysample

2) Rename company name directory:

from:

C:\github\MySample\app\src\main\java\
com\pstorli

to:

C:\github\MySample\app\src\main\java\
com\mycompany

3) New project directory should look like:

C:\github\MySample\app\src\main\java\
com\mycompany\mysample

h) Open MySample Project in Android Studio.

- 1) Edit → Find → Replace in files.
“**pstorli**” with “**mycompany**”
- 2) Edit → Find → Replace in files.
“**mvvm**” with “**mysample**”
- 3) Edit → Find → Replace in files, case sensitive.
“**MVVM**” with “**MySample**”
- 4) Select: **File** → **Invalidate Caches and Restart**

I) Update app icon.

- 1) Right click the res directory
- 2) Select New → Image Asset.
- 3) Ensure **Icon Type** is set to **Launcher Icons (Adaptive and Legacy)**.
- 4) Set Source Asset, Asset Type: **Image**
- 5) Set Source Asset, Path: **Path to your 512x512 icon.png**
- 6) Click **Next** → **Finish**

j) Integrate version control with github

- 1) Open **MySample** Project in Android Studio.
- 2) Share Project on github: **(Version Control System)**
Go to the menu bar and **select**
Git → **GitHub** → **Share Project on GitHub**

More Project “Code” details

It is **important** to understand that the UI thread must never be stopped or blocked, or your app will display the deadly “*app not responding*” dialog.

The app starts out in MainActivity.create, on the ui thread.

The first thing we do is instantiate the **ViewModel** class,

```
val viewModel: ViewModel by viewModels()
```

Which will be passed to our composeables on the constructor.

```
StartScreen (viewModel)
```

The **ViewModel** class is used as a central location to store project data.

The composeables use the viewModel to get data they need, and the repository uses the viewModel class as a place to store data perhaps retrieved in the background.

Which makes data sharing easy, because everyone knows where it is.

And, you get added protection for storing your data in the viewModel,

Your data can survive context switches, such as the phone being rotated from portrait to landscape.

Only quick things should be done on the ui thread,

such as asking the viewModel for data, no logic performed here.

What does happen is that if the viewModel data is modified, the ui layer, composeables, are notified and update themselves accordingly.

Logic can go in the viewModel class, but can also be delegated by creating viewModel children, such as these viewModel variables:

```
// Used to request long running data  
// Repository stuff, do long running things in the background.  
var repo = Repo (this)  
  
// Used to run coroutines in the background.  
var vh = VMHelper (this)  
  
// Used to persist data , even after app shut down.  
@Suppress("unused")  
var prefs = Prefs(app) // Preferences, initialize first
```

1) We launch a composable, **StartScreen**, in

com.mycompany.mysample.ui.composeables.screens.StartScreen.kt

- When the composable starts, it sets up to listen to any changes in the view models, backgroundColor variable.

With this line of code:

```
modifier = Modifier.fillMaxSize().background (viewModel.backColor)
```

- The buttons text color is also tied to the viewModels buttonColor

```
containerColor = Color (viewModel.buttonColor.value)
```

- It then also ties pressing the button on the main screen with executing the Method **toggleRunning** in class **VMHelper.kt**

viewModel.vh.toggleRunning()

The method **toggleRunning** toggles the running state of a background task.

Let's examine this in greater detail. First the code.

```
fun toggleRunning() {  
    // 1 toggle running state  
    viewModel.running = !viewModel.running  
  
    // Are we running?  
    var word: String  
  
    // 2 Are we running?  
    if (!viewModel.running) {  
        // 3 Make sure that background thread running.  
        // This task throws out random colors at random times.  
        // In this app it is simulating outside data trickling in.  
        back.startBackgroundTask()  
  
        // 4 Pressing button now starts background color task.  
        word = viewModel.app.getText(R.string.stop).toString()  
    }  
    else {  
        // 5 Pressing button now stops background color task.  
        word = viewModel.app.getText(R.string.start).toString()  
    }  
}
```

// 6 set button background color in a background thread.

// set the button text.

viewModel.buttonText = word

// 7 get word color in background.

back.getColorInBackground (word)}

In step **1**, when reverse the running state with a not !

In step **2**, we take slightly different paths if we are running or not.

In step **3**, we start background task in a co-routine using class CoHelper,
which stands for coroutineHelper

protected var ch = CoHelper (viewModel)

Note, in steps 4 and 5, we retrieve the color by calling our repository
on a background coroutine,

viewModel.buttonColor = **viewModel.repo**.getColorOfWord(start)

We would not normally do this, but wanted to show how to call a
background thread and get data from it back. When the background
coroutine is done, it notifies the viewModel

In step **4**, if we are running, set button text to stop and button color
to red.

In step **5**, If we are not running, set button text to Start and button color to
green.

In step **6**, set the button text.

In step **7**, *get word color in background.*

Notes:

- App execution starts at

`MainActivity.onCreate`, `MainActivity.kt`, also holds the `viewModel`, which holds project data.

`val viewModel: ViewModel by viewModels()`

- **Project root:**

`MVVM\app\src\main\java`

`app\kotlin+java\com\pstorli\mvvm`

- **To debug,**

- Left click mouse far left in file, breakpoint will appear as a red dot.
- Start program with bug icon, top left, of app or Run → Debug menu
- F7 step into, F8 step over, F9 run to next breakpoint
- when stopped at a breakpoint, you can view current state of variables.

- I highly suggest you download **WinMerge** to help compare projects, or versions of same project.

<https://winmerge.org/downloads>